

Progress Report

TrioTagaPayo — AMS_DBTC Project

Date:	April 18, 2026
Reporter:	James Neil M. Maano
Project:	TrioTagaPayo AMS DBTC
Report Period:	March 7, 2026 – April 18, 2026
Sprint / Period:	Current Sprint

STAND-UP REPORT

What was achieved this period?

- Set up the Backend folder/project structure
- Pushed code to GitHub repository and created a dedicated Backend branch
- Implemented clean code refactoring across backend files
- Updated and refined pagination functionality via PaginationHelper
- **Added QR Code Attendance System (NEW)**
 - QRSession and QRScan models added
 - IQRSessionRepository interface defined with token lookup, scan tracking, and active session queries
 - QRSessionRepository implementation with full DB query support
 - IQRService interface designed with GenerateAsync, ScanAsync, DeactivateAsync, GetActiveSessionsAsync
 - ApplicationDbContext updated: QRSessions and QRScans DbSet, relationships, and unique scan index
- **Integrated EmailJS Email Notification System (NEW)**
 - EmailJSHelper implemented — sends attendance notification emails via EmailJS REST API
 - EmailJSOptions configuration class added and wired to appsettings
 - HttpClient registered with AddHttpClient<EmailJSHelper>() to avoid double-registration
 - Email failure is isolated — will never crash the attendance save flow
- **Enhanced Attendance Repository (NEW)**
 - IAttendanceFilterRepository added — supports date-range filtering by course
 - IAttendanceBulkRepository added — supports bulk ID-based lookups
 - AttendanceRepository implements all three interfaces
- **Added Global Exception Middleware (NEW)**
 - ExceptionMiddleware catches all unhandled exceptions and returns structured JSON error responses
 - Maps exception types to proper HTTP status codes (404, 400, 401, 500)
- **Program.cs fully configured (NEW)**
 - All repositories and services registered with proper scoped lifetimes
 - JWT Bearer authentication configured with zero clock skew
 - Swagger/OpenAPI with Bearer security definition enabled
 - CORS policy configured for AllowAll
 - Admin seeding on startup via IAuthService.SeedAdminAsync()

What is the focus today?

- Complete QRService implementation (GenerateAsync, ScanAsync, DeactivateAsync)
- Register IQRService and IQRSessionRepository in Program.cs
- Create QRController with Generate, Scan, Deactivate, and GetActiveSessions endpoints
- Test QR scan flow end-to-end: token generation, expiry, duplicate scan prevention
- Verify EmailJS email delivery for attendance notifications

What blockers exist?

- QRService implementation not yet complete — ScanAsync logic pending
- IQRService not yet registered in Program.cs DI container
- EmailJS private key stored in user secrets — needs verification in staging
- QR scan flow needs integration testing before it can be used in production

RETROSPECTIVE

What went well? (Wins/Successes)

- QR-based attendance feature is well-scoped with clean separation of models, repo, and service layers
- EmailJS integration was added without coupling it tightly to the attendance save — failure-safe design
- Repository pattern is now more flexible with the addition of filter and bulk interfaces
- Global exception middleware eliminates boilerplate try/catch across all controllers
- Backend folder structure and Git branching remain organized
- Pagination helper is now reusable across all services

What didn't go well? (Problems/Challenges)

- QRService implementation is still incomplete — ScanAsync and GenerateAsync need finishing
- Program.cs is missing QRService and QRSessionRepository DI registrations
- Some clean code refactoring still ongoing
- Testing coverage for QR flow and email notifications is not yet done

What should we start doing? (Improvements)

- Write integration tests for QR scan flow (token generation, expiry, duplicate prevention)
- Add IQRService and IQRSessionRepository to Program.cs DI
- Test EmailJS email delivery with real student email addresses
- Document API endpoints in Swagger with XML summary comments

What should we stop doing? (Waste/Inefficiencies)

- Avoid leaving DI registrations incomplete after implementing new services
- Avoid committing untested code changes — test after each feature addition

What should we continue doing? (Best Practices)

- Maintain organized folder structure and feature-branch Git workflow
- Continue using repository pattern with interface-based DI for testability
- Keep email and notification failures isolated from core business logic
- Continue consistent progress updates and stand-up check-ins

Submitted by:

James Neil M. Maano

TrioTagaPayo Team

March 7, 2026 – April 18, 2026